

Azure Cloud Honeypot & SOC Lab

Author: Jacob Flores

Date Submitted: 09/08/2026

Link to git repository:

<https://github.com/JacobFloress/Cloud-Security-Honeypot-SOC-Detection-Pipeline>

1. Summary

In this project, I built a complete Security Operations Center (SOC) pipeline inside Microsoft Azure. Basically, I deployed a Windows VM on purpose so that it would be attacked (a honeypot), captured the real attack traffic hitting it, centralized the logs, enriched them with geolocation data, and then visualized where the attackers were actually coming from. My main goal was to understand how a detection pipeline is built from the ground up, using free-tier Azure resources, and to see for myself how quickly an exposed machine gets discovered on the public internet.

The pipeline I stood up consists of the following pieces:

1. An internet-exposed Windows 10 VM configured to attract unsolicited login attempts
2. Centralized log forwarding into a Log Analytics Workspace (LAW)
3. A Microsoft Sentinel (SIEM) instance connected to that workspace
4. KQL-based analysis of failed authentication attempts (Event ID 4625)
5. Geolocation enrichment of the attacker IP addresses
6. A geographic visualization showing where the attacks originated

Within just a few hours of deployment, the honeypot had already recorded thousands of failed login attempts from sources in dozens of countries. This alone was a pretty concrete demonstration of how fast unprotected infrastructure gets found and targeted once it's on the public internet.

2. Architecture - Security Operations Center (SOC)

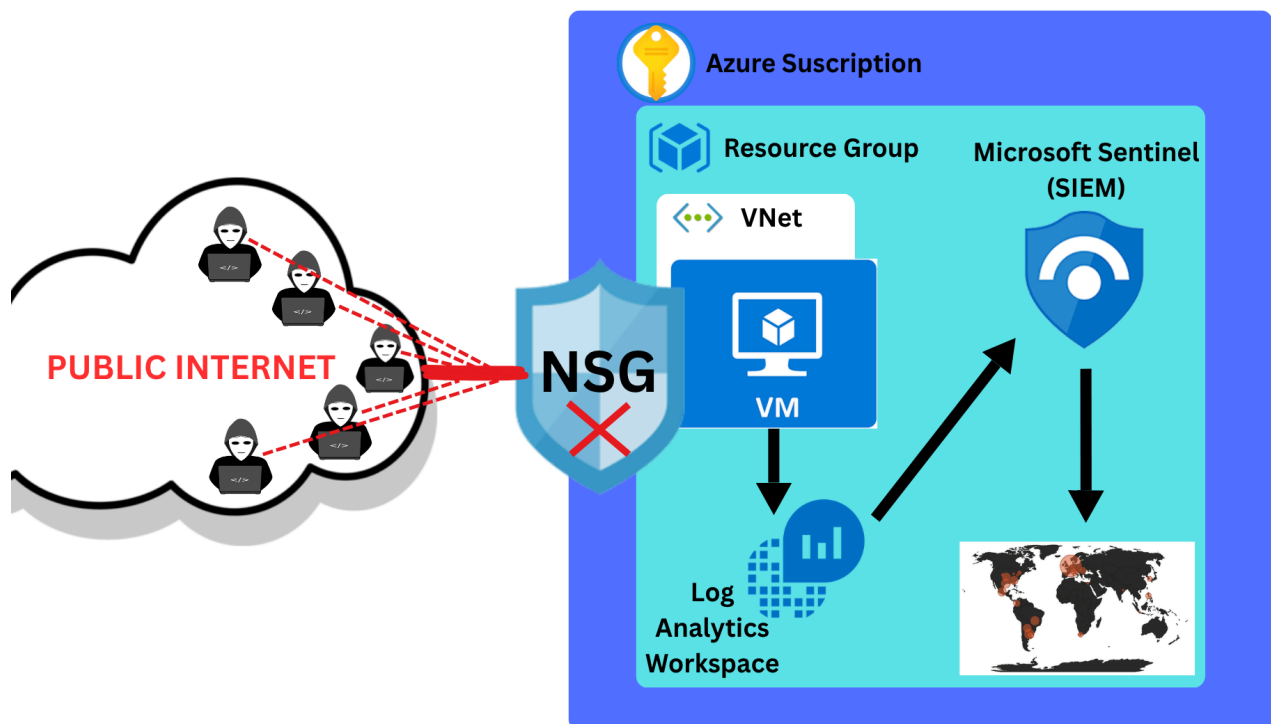


Figure 1: SOC Pipeline Architecture

3. Build Process

3.1 Subscription and networking

I started by creating a free Azure subscription, a resource group, and a virtual network/subnet to host the lab in.

3.2 Honeypot VM

Next, I deployed a Windows 10 VM into that virtual network. To make it as attractive as possible to opportunistic internet scanners, I did the following:

- Replaced the default Network Security Group rule with one that allows all inbound traffic, on all ports, for all protocols
- Disabled the in-guest Windows Defender Firewall entirely
- Verified that it was actually reachable from an external network with a basic ping test

3.3 Baseline log inspection

Before configuring any log forwarding, I wanted to confirm the logging behavior locally first. So I intentionally triggered a few failed logins (for example, trying to log in as an employee account that doesn't exist) and checked that they showed up in the local Event Viewer, under Windows Logs → Security, as Event ID 4625 ("An account failed to log on"). This confirmed that the VM was logging what I needed it to before I built anything on top of it.

3.4 Centralized logging pipeline

Once I confirmed the local logging worked, I moved on to centralizing it. I did this by:

- Creating a Log Analytics Workspace to act as the central log repository
- Deploying a Microsoft Sentinel instance and connecting it to that workspace
- Installing the Windows Security Events via AMA data connector
- Creating a Data Collection Rule (DCR) scoped specifically to the honeypot VM
- Confirming that the Azure Monitor Agent installed correctly and that logs actually started flowing into the workspace (this took about 20–40 minutes before the first events showed up)

3.5 Querying with KQL

Once logs were flowing, I used Kusto Query Language (KQL) inside Log Analytics to filter and inspect the events. For example:

```
SecurityEvent
| where EventID == 4625
| project TimeGenerated, Computer, AccountName, IpAddress
| order by TimeGenerated desc
```

Within 24 hours, the honeypot had already logged thousands of failed login attempts from public IP addresses all over the world, attackers trying common usernames like `admin` and `administrator` against the exposed RDP-adjacent surface.

4. Challenge: Sentinel Watchlist and Workbook outage

The standard version of this lab relies on two native Sentinel features to finish the pipeline:

- A Watchlist, an uploaded CSV mapping IP ranges (CIDR blocks) to city/country/lat-long, used to enrich logs with geolocation via the `ipv4_lookup()` KQL plugin
- A Workbook, a Sentinel dashboard that visualizes the enriched data on an interactive world map

While building this, I ran into a problem. Both the Watchlist and Workbook blades in the Azure/Defender portal were affected by what turned out to be a known, actively-reported platform issue: clicking into either page just redirected back to Settings → Microsoft Sentinel → SIEM Workspaces, in a loop, no matter the account permissions, browser, or session state I tried. After digging into it, I confirmed this was a broader Sentinel/Defender-portal migration issue affecting multiple users at the time, not something wrong with my own deployment.

Rather than wait around for a platform-side fix, I decided to re-implement the same functionality myself, using tools that don't depend on the broken UI.

5. Workaround: replacing the Watchlist

A Sentinel Watchlist is really just a managed lookup table underneath. For this reason, I realized I could get the same result using KQL's built-in `externaldata()` operator, which fetches a CSV from any URL at query time, no Sentinel feature required.

Here is what I did:

1. Provisioned an Azure Storage Account (in the same resource group) with a Blob container
2. Enabled anonymous blob-level read access, scoped to just that container and not the whole account, and uploaded the GeoIP CSV
3. Queried the hosted CSV directly from Log Analytics, joining it against the attacker IPs using the `ipv4_lookup()` plugin (the same CIDR-matching function the Watchlist-based approach would have used):

```
let geoip = externaldata(network:string, latitude:real, longitude:real,
cityname:string, countryname:string)
["https://<storageaccount>.blob.core.windows.net/geodata/geoip-summarized.csv"]
with (format="csv", ignoreFirstRecord=true);
SecurityEvent
| where EventID == 4625
| project TimeGenerated, Computer, AttackerIP = IpAddress
| evaluate ipv4_lookup(geoip, AttackerIP, network)
| summarize AttackCount = count() by cityname, countryname, latitude, longitude
| order by AttackCount desc
```

This produced the exact same output the native Watchlist join would have given me: each failed-login event enriched with the attacker's city, country, and coordinates.

6. Workaround: replacing the Workbook map

Since the Workbook editor was inaccessible too, I couldn't use Sentinel's built-in map visualization. Instead, I exported the aggregated, geo-enriched query results directly from Log Analytics (Export → CSV) and rendered them as a standalone interactive world map on my own. Attacker location shown as a

bubble, with bubble size and color intensity scaled to attack volume, and hover tooltips showing the exact city, country, and count.

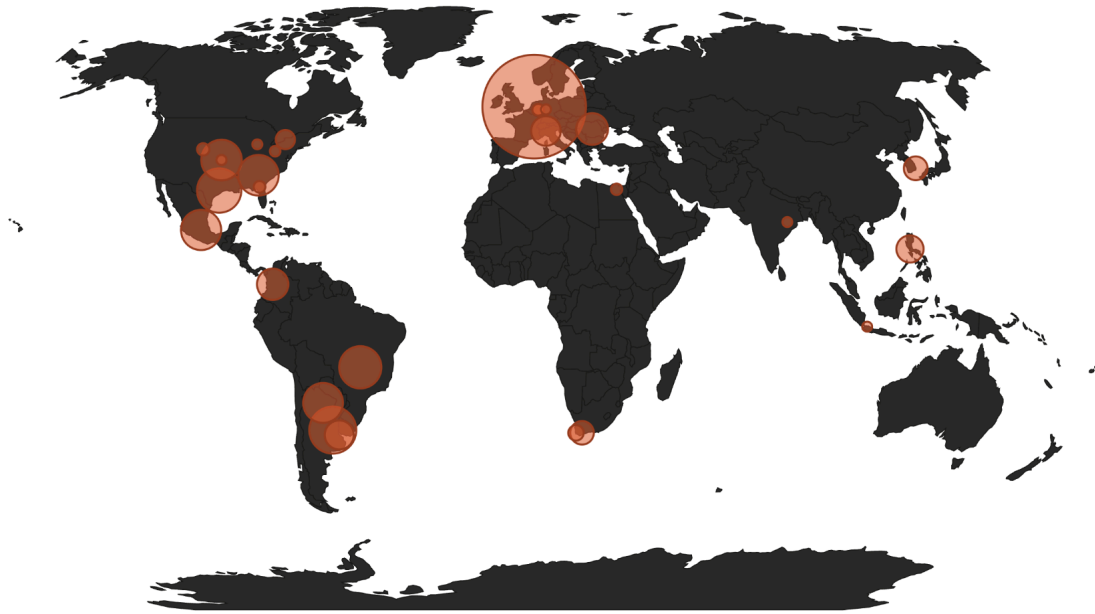


Figure 2: *Attacker Distribution Map (24-hour window)*

Here is the raw output from that query before I organized it into the table below:

cityname	countryname	latitude	longitude	AttackCount
Maarn	Netherlands	52.0672	5.3781	11949
San Nicolás de los Arroyos	Argentina	-33.3245	-60.22	1928
Katy	United States	29.7388	-95.8309	1662
Aparecida de Goiania	Brazil	-16.8428	-49.2468	1472
-	United States	37.7548	-97.8277	1329
-	United States	33.7485	-84.3871	1329
Iztacalco	Mexico	19.3958	-99.0959	1298
Pampa de los Guanacos	Argentina	-26.1944	-62	1265
-	Romania	45.9968	24.997	688
Manizales	Colombia	5.072	-75.515	663
Milan	Italy	45.4722	9.1922	562
-	Argentina	-34.6022	-58.3845	532
General Trias	Philippines	14.3842	120.884	437

Jung-gu	South Korea	35.572	129.3302	287
Swellendam	South Africa	-34.0325	20.4388	280
Lockport	United States	43.1612	-78.6946	138
Cape Town	South Africa	-34.0541	18.4791	54
Fort Collins	United States	40.5748	-105.0951	14
Düsseldorf	Germany	51.2087	6.7791	9
Düsseldorf	Germany	51.2184	6.7734	9
Lake Park	United States	30.6944	-83.1698	9
Cairo	Egypt	30.0588	31.2268	7
Saint Clairsville	United States	40.0778	-80.9788	3
-	Germany	51.2993	9.491	1
Pengarengan	Indonesia	-6.1234	106.8013	1
Sambalpur	India	21.4668	83.9764	1
-	United States	37.751	-97.822	1
Chicago	United States	41.8972	-87.6196	1

Table 1: Log Analytics query results (24-hour window)

7. Results

The final enriched dataset showed failed login attempts coming from at least 15 different countries within the observation window. There were large volumes from the Netherlands, Argentina, the United States, Brazil, and Mexico, but also single-digit probing attempts from countries like Egypt, India, and Indonesia. This was interesting to see because it shows both high-volume automated scanning and low-volume, more opportunistic attempts hitting the same exposed host at the same time.

Here is the summary view I pulled together from these results:

Metric	Value
Failed login events captured (Event ID 4625)	6,000+ within 24 hours; tens of thousands over the observation period
Distinct source locations identified	15+ countries
Top source	Netherlands (11,949 failed attempts from a single resolved location)
Time to first external discovery	Under 24 hours of public exposure

Table 2: Attack summary (24-hour window)

8. Results After 7 Days

I decided to extend the time range to the last 7 days to see if the picture would change. It did, quite a bit. The total count reached 899,072 failed login attempts across the observation window.

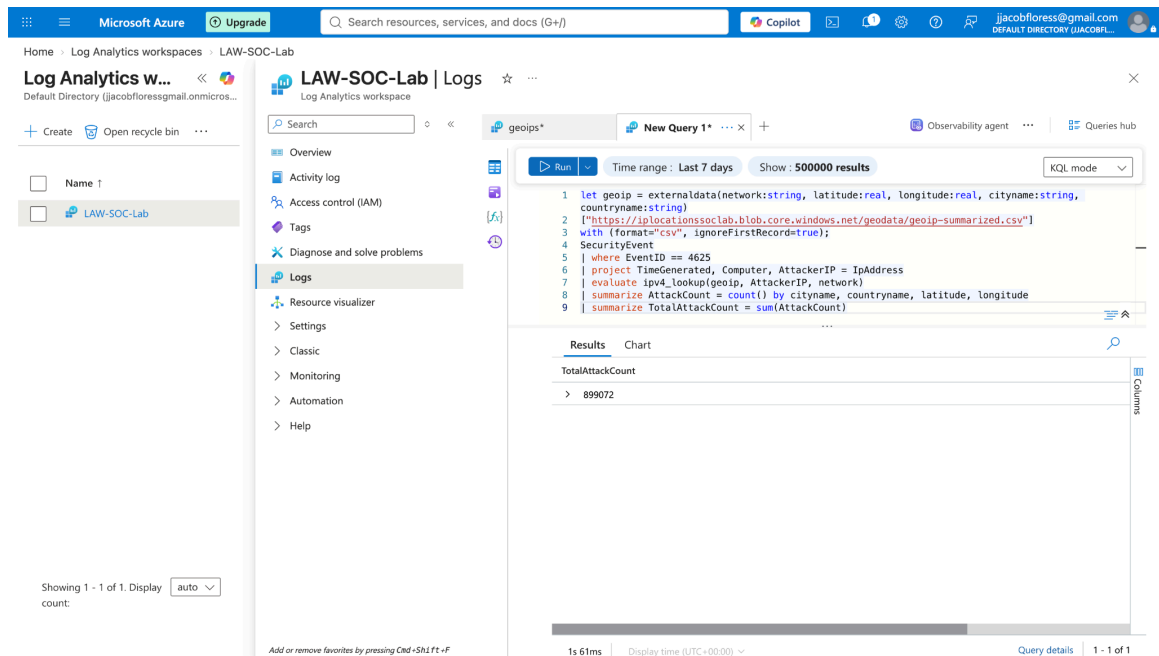


Figure 5: Total attacker count over 7 days

The number of distinct attacker locations went from the 15 countries I saw in the first 24 hours to 352 unique city/country combinations, so the first day was really only scratching the surface.

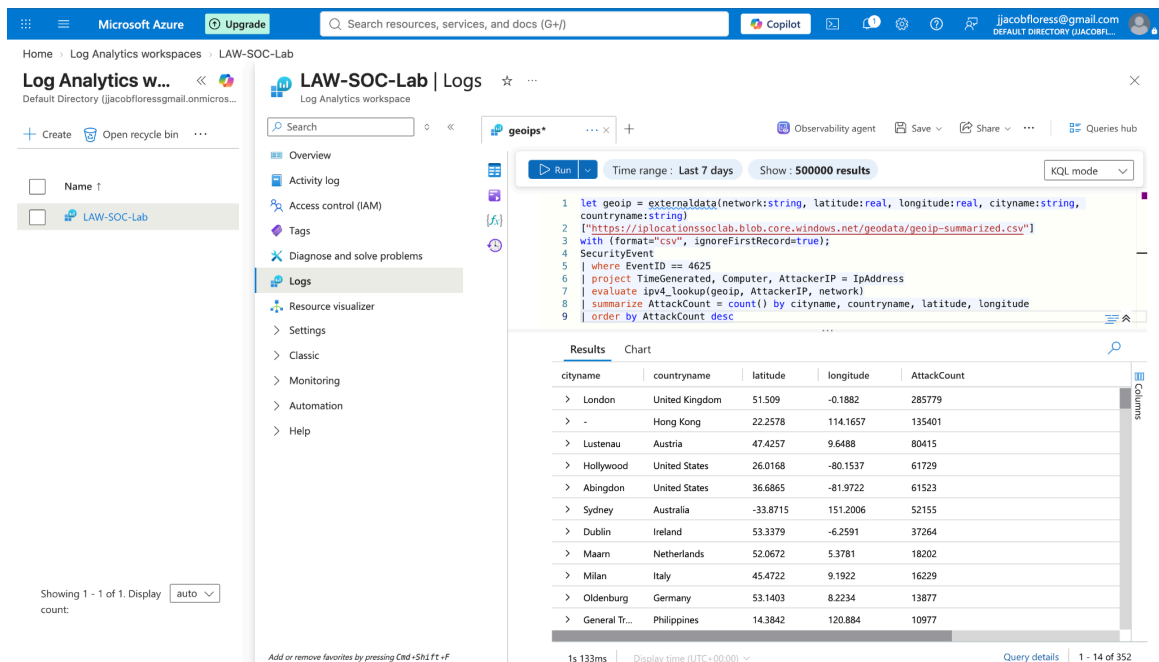


Figure 6: 352 unique attacker locations (7-day window)

London, United Kingdom, ended up as the single largest source by far, with 285,779 failed login attempts, more than double the next closest one. Hong Kong followed with 135,401, and Lustenau, Austria came in third with 80,415. After that, the numbers drop off more gradually (Hollywood and Abingdon in the US, Sydney, Dublin), all landing in a similar mid-range.

What I found interesting is that this looks very different from the early snapshot. In 24 hours, the Netherlands was the top source. Over the full week, that same location (Maarn) only reached 18,202 attempts, well behind London and Hong Kong. This tells me that early results can be misleading if you only look at a short window, since the sources that show up first aren't necessarily the ones that generate the most volume in the long run.

The query itself still ran in about a second (1s 133ms) even against the full 7-day window, so the `externaldata()/ipv4_lookup()` workaround holds up fine at this scale.

9. Skills demonstrated

Through this project, I got hands-on practice with:

- Azure resource provisioning: resource groups, virtual networks, VMs, storage accounts
- Network security group and OS-level firewall configuration
- Log Analytics Workspace and Microsoft Sentinel (SIEM) deployment and configuration
- Windows Security event log analysis (Event Viewer, Event ID interpretation)
- KQL: filtering, projection, aggregation, and the `externaldata()` and `ipv4_lookup()` plugins
- Log enrichment techniques for threat intelligence and geolocation correlation
- Troubleshooting and adapting when a managed platform feature is unavailable, including building an equivalent pipeline out of lower-level primitives
- Basic Azure Blob Storage configuration, including scoped anonymous access

10. Cleanup

After finishing the lab, I deprovisioned all the resources (VM, storage account, Log Analytics Workspace, Sentinel instance, resource group) to avoid ongoing Azure charges. In a real production environment, this same pipeline, the log forwarding, the enrichment, and the dashboarding, would be left running continuously, and it would be backed by an automatically-updating threat intelligence feed instead of the static CSV I used here.

11. Conclusion

This project taught me a lot about how a SOC pipeline actually comes together, piece by piece, and how quickly an exposed machine gets found once it's sitting on the public internet. Running into the Watchlist/Workbook outage ended up being one of the more valuable parts of the project, since it forced me to understand what those features were actually doing underneath the UI, instead of just clicking through them. I would really like to extend this in the future by integrating a live, automatically-updating threat intelligence feed instead of a static CSV, and maybe by adding more exposed services to the honeypot to see how the attack patterns differ.